import libraries

In [1]:
```
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.simplefilter('ignore')
```

In [2]:
```
# ============================================================
# INTRODUCTION
# ============================================================
introduction = """
INTRODUCTION
------------
This analysis uses a household dataset containing ID, Age, Gender, Educat
Monthly_Income, and Monthly_Expenditure. The data includes missing values
requiring cleaning before analysis. Using Python libraries such as Pandas
Matplotlib, and Seaborn, the study performs data importation, missing-val
outlier detection via the IQR method, descriptive statistics, and data ma
(selection, sorting, mutation, grouping, filtering). Visualizations—histo
boxplots, and scatterplots—are used to explore income distribution, educa
age–income relationships, and outliers. Integrated analysis further exami
and expenditure by education and gender, identifies households with savin
and summarizes findings by gender. The purpose is to generate evidence-ba
household income, expenditure, savings, and demographic patterns.
"""
print(introduction)

# ============================================================
# OBJECTIVES
# ============================================================
objectives = """
OBJECTIVES
----------
1. Import, inspect, clean, and explore the household dataset.
2. Handle missing values and detect outliers in Monthly_Income.
3. Perform Pandas operations: select, sort, mutate, group, and filter.
4. Visualize income distribution, education counts, outliers, and Age–Inc
5. Analyze average income and expenditure by Education_Level and Gender.
6. Identify high-savings households and summarize key findings by gender.
"""
print(objectives)
```

```
INTRODUCTION
------------
This analysis uses a household dataset containing ID, Age, Gender, Educati
on_Level,
Monthly_Income, and Monthly_Expenditure. The data includes missing values
and outliers,
requiring cleaning before analysis. Using Python libraries such as Pandas,
NumPy,
Matplotlib, and Seaborn, the study performs data importation, missing-valu
e imputation,
outlier detection via the IQR method, descriptive statistics, and data man
ipulation
(selection, sorting, mutation, grouping, filtering). Visualizations—histog
rams, bar charts,
boxplots, and scatterplots—are used to explore income distribution, educat
ion levels,
age–income relationships, and outliers. Integrated analysis further examin
es average income
and expenditure by education and gender, identifies households with saving
s above KES 20,000,
and summarizes findings by gender. The purpose is to generate evidence-bas
ed insights into
household income, expenditure, savings, and demographic patterns.


OBJECTIVES
----------
1. Import, inspect, clean, and explore the household dataset.
2. Handle missing values and detect outliers in Monthly_Income.
3. Perform Pandas operations: select, sort, mutate, group, and filter.
4. Visualize income distribution, education counts, outliers, and Age–Inco
me relationship.
5. Analyze average income and expenditure by Education_Level and Gender.
6. Identify high-savings households and summarize key findings by gender.
```

data importation

In [3]:
```
df = pd.read_csv('dataset.csv')
df.head()
```

Out[3]:

	ID	Age	Income	Expenditure	Household_Size	Education_Level	Gender
0	1	62.0	32592.0	24334.0	1.0	Secondary	Male
1	2	30.0	114530.0	82539.0	2.0	Primary	Female
2	3	24.0	21905.0	11773.0	2.0	Primary	Male
3	4	63.0	109924.0	59158.0	7.0	College	Male
4	5	32.0	109506.0	64181.0	7.0	Secondary	Male

print first 10 observation

In [4]:
```
print("First 10 observations:\n", df.head(10))
```

```
First 10 observations:
    ID   Age    Income  Expenditure  Household_Size Education_Level  Gende
r
0    1  62.0   32592.0      24334.0             1.0       Secondary    Male
1    2  30.0  114530.0      82539.0             2.0         Primary  Female
2    3  24.0   21905.0      11773.0             2.0         Primary    Male
3    4  63.0  109924.0      59158.0             7.0         College    Male
4    5  32.0  109506.0      64181.0             7.0       Secondary    Male
5    6   NaN   31396.0      18899.0             2.0         College  Female
6    7  60.0   52671.0      39076.0             1.0         Primary  Female
7    8  27.0   90357.0      70638.0             5.0         College    Male
8    9  26.0   24006.0      18226.0             4.0         Primary    Male
9   10  28.0  650000.0      42821.0             5.0         College    Male
```

print last 10 observation

In [5]:
```
print("\nLast 10 observations:\n", df.tail(10))
```

```
Last 10 observations:
      ID   Age    Income  Expenditure  Household_Size Education_Level  Gen
der
90   91  56.0  115978.0      74840.0             4.0         College  Fema
le
91   92  53.0   21804.0      12743.0             7.0      University    Ma
le
92   93  51.0   34728.0      28226.0             NaN         Primary    Ma
le
93   94  63.0   74179.0      59101.0             3.0       Secondary    Ma
le
94   95  38.0   67689.0      36191.0             6.0         College  Fema
le
95   96  46.0   54471.0      30009.0             7.0         Primary  Fema
le
96   97  23.0  116176.0      98745.0             1.0         College    Ma
le
97   98  63.0   26993.0      20290.0             1.0       Secondary    Ma
le
98   99  23.0   99439.0      54236.0             3.0      University    Ma
le
99  100  58.0   46569.0      33985.0             8.0         College    Ma
le
```

Dataset structures

In [6]:
```
print("\nDataset structure:")
print(df.info())
print("\nDimensions:", df.shape)
print("\nData types:\n", df.dtypes)
```

```
Dataset structure:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   ID               100 non-null    int64
 1   Age              98 non-null     float64
 2   Income           98 non-null     float64
 3   Expenditure      98 non-null     float64
 4   Household_Size   99 non-null     float64
 5   Education_Level  100 non-null    object
 6   Gender           100 non-null    object
dtypes: float64(4), int64(1), object(2)
memory usage: 5.6+ KB
None

Dimensions: (100, 7)

Data types:
 ID                  int64
Age                float64
Income             float64
Expenditure        float64
Household_Size     float64
Education_Level     object
Gender              object
dtype: object
```

check for missing values

In [7]:
```
print("\nMissing values per variable:\n", df.isnull().sum())
```

```
Missing values per variable:
 ID                 0
Age                2
Income             2
Expenditure        2
Household_Size     1
Education_Level    0
Gender             0
dtype: int64
```

Handle missing numerical values using median (or mean)

In [8]:
```
# --- Normalize column names (fixes KeyError) ---
df.columns = df.columns.str.strip()          # remove leading/trailing sp
df.columns = df.columns.str.replace(' ', '_') # spaces -> underscores

# Map common variants to the expected names
rename_map = {
    'monthly_income': 'Monthly_Income',
    'Monthly Income': 'Monthly_Income',
    'Income': 'Monthly_Income',
    'Monthly_Income_KES': 'Monthly_Income',
```

```
    'monthly_expenditure': 'Monthly_Expenditure',
    'Monthly Expenditure': 'Monthly_Expenditure',
    'Expenditure': 'Monthly_Expenditure',
    'education_level': 'Education_Level',
    'Education Level': 'Education_Level',
    'Education': 'Education_Level',
    'gender': 'Gender',
    'age': 'Age',
    'id': 'ID',
}
df = df.rename(columns={c: rename_map.get(c, c) for c in df.columns})

# Verify all required columns exist
required = ['ID', 'Age', 'Gender', 'Education_Level',
            'Monthly_Income', 'Monthly_Expenditure']
missing = [c for c in required if c not in df.columns]
if missing:
    raise ValueError(
        f"These columns are still missing: {missing}\n"
        f"Available columns: {df.columns.tolist()}"
    )

print("Columns after cleaning:", df.columns.tolist())
```

```
Columns after cleaning: ['ID', 'Age', 'Monthly_Income', 'Monthly_Expenditu
re', 'Household_Size', 'Education_Level', 'Gender']
```

imputing median to continuos values

In [9]:
```
for col in ['Age', 'Monthly_Income', 'Monthly_Expenditure']:
    if df[col].isnull().any():
        df[col].fillna(df[col].median(), inplace=True)
```

Handle missing categorical values using mode

In [10]:
```
for col in ['Gender', 'Education_Level']:
    if df[col].isnull().any():
        df[col].fillna(df[col].mode()[0], inplace=True)
```

Detect outliers in Monthly_Income using IQR

In [11]:
```
Q1 = df['Monthly_Income'].quantile(0.25)
Q3 = df['Monthly_Income'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = df[(df['Monthly_Income'] < lower_bound) | (df['Monthly_Income'
print(f"\nNumber of outliers in Monthly_Income: {len(outliers)}")
```

```
Number of outliers in Monthly_Income: 2
```

Remove outliers in Monthly_Income using IQR method

In [12]:
```
before = df.shape[0]

# Keep only rows within the bounds
df = df[(df['Monthly_Income'] >= lower_bound) &
        (df['Monthly_Income'] <= upper_bound)].copy()

after = df.shape[0]

print(f"Outlier bounds: lower = {lower_bound:.2f}, upper = {upper_bound:.
print(f"Rows before removal: {before}")
print(f"Rows removed (outliers): {before - after}")
print(f"Rows after removal: {after}")
```

```
Outlier bounds: lower = -24218.12, upper = 168872.88
Rows before removal: 100
Rows removed (outliers): 2
Rows after removal: 98
```

Descriptive statistics

In [13]:
```
print("\nDescriptive statistics (numerical):\n", df.drop(columns=["ID"]).
```

```
Descriptive statistics (numerical):
              Age  Monthly_Income  Monthly_Expenditure  Household_Size
count  98.000000       98.000000            98.000000       97.000000
mean   45.295918    69999.295918         51228.000000        4.257732
std    13.969930    29762.143905         52623.457826        2.242015
min    22.000000    18772.000000         10690.000000        1.000000
25%    34.000000    47951.000000         28415.250000        2.000000
50%    47.000000    72508.000000         44951.500000        4.000000
75%    56.000000    95472.500000         62875.000000        6.000000
max    96.000000   117871.000000        520000.000000        8.000000
```

Mean, median, variance, std of Monthly_Income

In [14]:
```
mi = df['Monthly_Income']
print(f"\nMonthly_Income -> Mean: {mi.mean():.2f}, Median: {mi.median():.
      f"Variance: {mi.var():.2f}, Std: {mi.std():.2f}")
```

```
Monthly_Income -> Mean: 69999.30, Median: 72508.00, Variance: 885785209.8
2, Std: 29762.14
```

DATA MANIPULATION USING PANDAS

data selection

In [15]:
```
selected = df[['ID', 'Age', 'Monthly_Income', 'Gender']]
print("\nSelected columns (first 5):\n", selected.head())
```

```
Selected columns (first 5):
    ID   Age  Monthly_Income  Gender
0   1  62.0         32592.0    Male
1   2  30.0        114530.0  Female
2   3  24.0         21905.0    Male
3   4  63.0        109924.0    Male
4   5  32.0        109506.0    Male
```

sorting of Data

In [16]:
```
df_sorted = df.sort_values(by='Monthly_Income', ascending=False)
print("\nTop 5 households by Monthly_Income:\n", df_sorted.head())
```

```
Top 5 households by Monthly_Income:
     ID   Age  Monthly_Income  Monthly_Expenditure  Household_Size  \
21  22  54.0        117871.0              76978.0             3.0
17  18  47.0        117061.0              92998.0             1.0
47  48  42.0        116548.0              52881.0             2.0
76  77  27.0        116453.0              83743.0             4.0
59  60  61.0        116235.0              95348.0             3.0

   Education_Level  Gender
21         Primary  Female
17       Secondary    Male
47         Primary    Male
76       Secondary    Male
59       Secondary    Male
```

mutate annual income to dataset

In [17]:
```
df['Annual_Income'] = df['Monthly_Income'] * 12
df.head()
```

Out[17]:

	ID	Age	Monthly_Income	Monthly_Expenditure	Household_Size	Education_Leve
0	1	62.0	32592.0	24334.0	1.0	Secondar
1	2	30.0	114530.0	82539.0	2.0	Primar
2	3	24.0	21905.0	11773.0	2.0	Primar
3	4	63.0	109924.0	59158.0	7.0	Colleg
4	5	32.0	109506.0	64181.0	7.0	Secondar

avarage income by education level

In [18]:
```
avg_income_edu = df.groupby('Education_Level')['Monthly_Income'].mean()
print("\nAverage Monthly_Income by Education_Level:\n", avg_income_edu)
```

```
Average Monthly_Income by Education_Level:
 Education_Level
College       66588.857143
Primary       68273.222222
Secondary     78069.913043
University    67822.900000
Name: Monthly_Income, dtype: float64
```

check for high income

In [19]:
```
high_income = df[df['Monthly_Income'] > 50000]
print(f"\nHouseholds with Monthly_Income > KES 50,000: {len(high_income)}
```

```
Households with Monthly_Income > KES 50,000: 70
```

households with age > 40

In [20]:
```
age_gt40 = df[df['Age'] > 40]
print(f"Households with Age > 40: {len(age_gt40)}")
```

```
Households with Age > 40: 58
```

avarage montly income expenditure by Gender

In [21]:
```
avg_exp_gender = df.groupby('Gender')['Monthly_Expenditure'].mean()
print("\nAverage Monthly_Expenditure by Gender:\n", avg_exp_gender)
```

```
Average Monthly_Expenditure by Gender:
 Gender
Female    44680.955556
Male      56786.811321
Name: Monthly_Expenditure, dtype: float64
```

assign savings to dataset

In [22]:
```
df['Savings'] = df['Monthly_Income'] - df['Monthly_Expenditure']
df.head()
```

Out[22]:

	ID	Age	Monthly_Income	Monthly_Expenditure	Household_Size	Education_Leve
0	1	62.0	32592.0	24334.0	1.0	Secondar
1	2	30.0	114530.0	82539.0	2.0	Primar
2	3	24.0	21905.0	11773.0	2.0	Primar
3	4	63.0	109924.0	59158.0	7.0	Colleg
4	5	32.0	109506.0	64181.0	7.0	Secondar

DATA VISUALIZATION

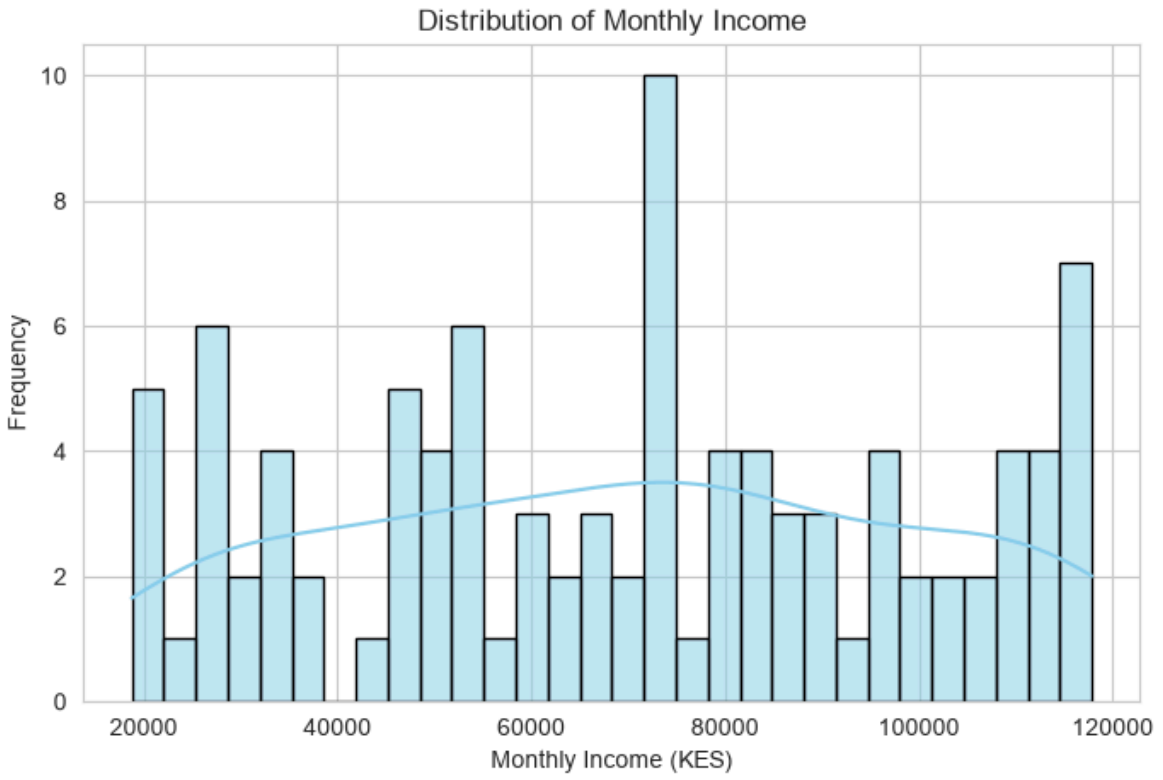
In [23]:
```
sns.set_style("whitegrid")
```

histogram

In [40]:
```
fig, ax = plt.subplots(figsize=(8, 5))

sns.histplot(
    data=df,
    x='Monthly_Income',
    bins=30,
    color='skyblue',
    edgecolor='black',
    kde=True,
    ax=ax,
)

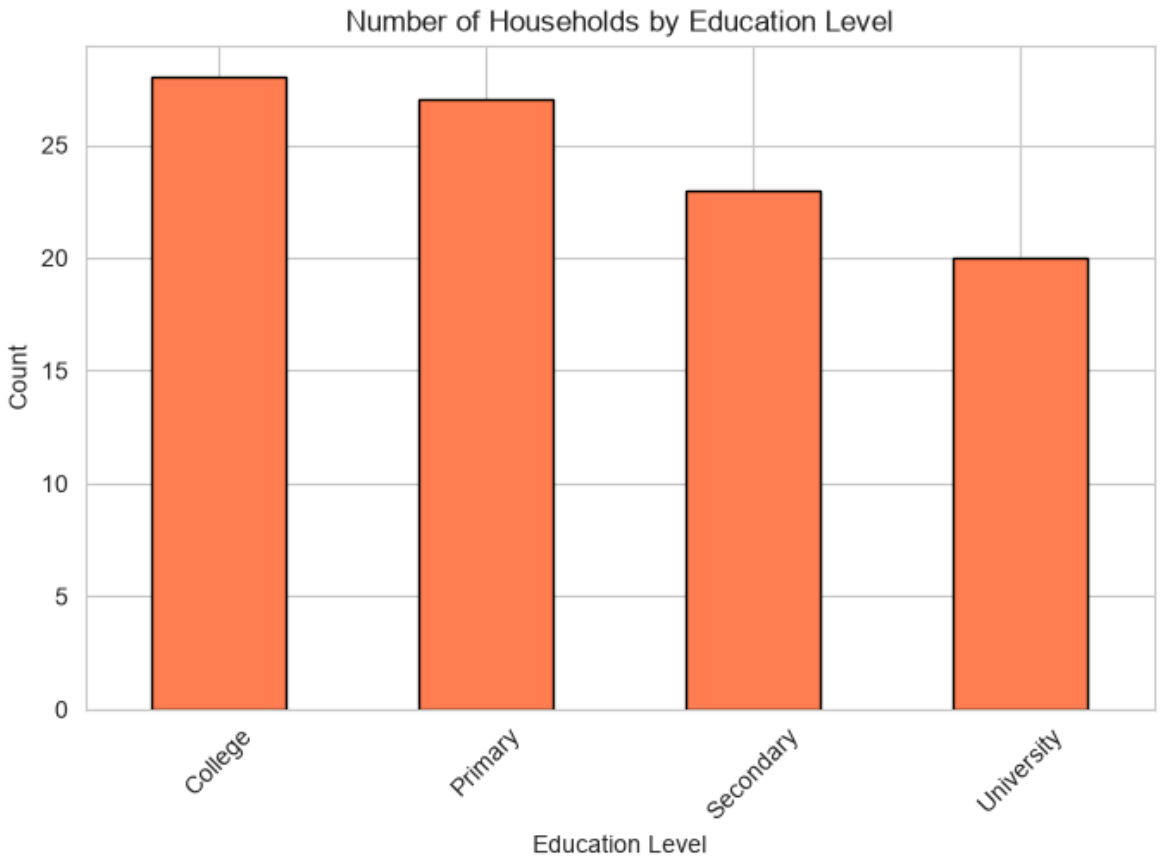
ax.set(
    title='Distribution of Monthly Income',
    xlabel='Monthly Income (KES)',
    ylabel='Frequency',
)
plt.show()
```



Bar chart

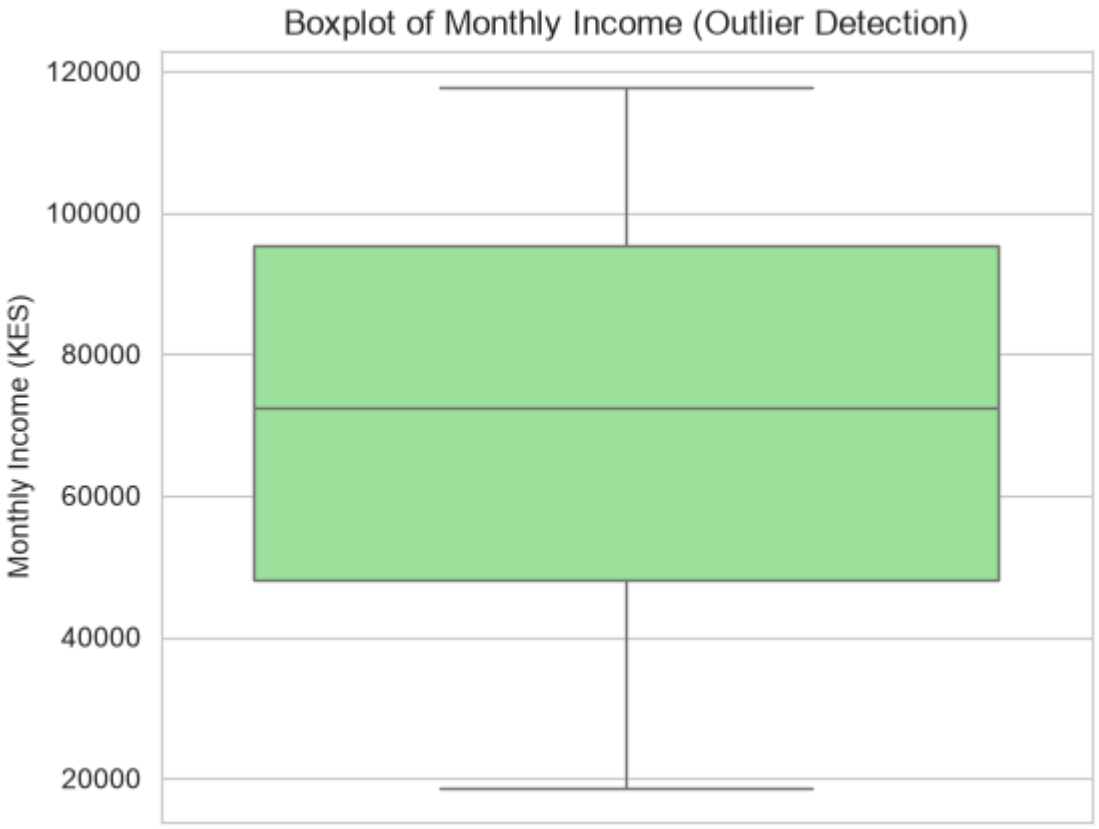
In [25]:
```
plt.figure(figsize=(8,5))
df['Education_Level'].value_counts().plot(kind='bar', color='coral', edge
```

```
plt.title('Number of Households by Education Level')
plt.xlabel('Education Level')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()
```



Boxplot

In [27]:
```
plt.figure(figsize=(6,5))
sns.boxplot(y=df['Monthly_Income'], color='lightgreen')
plt.title('Boxplot of Monthly Income (Outlier Detection)')
plt.ylabel('Monthly Income (KES)')
plt.show()
```



Scatterplot

In [28]:
```
plt.figure(figsize=(8,5))
plt.scatter(df['Age'], df['Monthly_Income'], alpha=0.6,color='purple')
plt.title('Age vs Monthly Income')
plt.xlabel('Age (years)')
plt.ylabel('Monthly Income (KES)')
plt.show()
```



INTEGRATED DATA ANALYSIS

Average Income & Expenditure by Education_Level

In [29]:
```
edu_summary = df.groupby('Education_Level')[['Monthly_Income', 'Monthly_E
print("\nAverage Income & Expenditure by Education_Level:\n", edu_summary
```

```
Average Income & Expenditure by Education_Level:
                 Monthly_Income  Monthly_Expenditure
Education_Level
College            66588.857143         60481.857143
Primary            68273.222222         46859.740741
Secondary          78069.913043         52752.717391
University         67822.900000         42416.325000
```

Gender with highest average income

In [30]:
```
gender_income = df.groupby('Gender')['Monthly_Income'].mean()
print("\nAverage Income by Gender:\n", gender_income)
print("Gender with highest average income:", gender_income.idxmax())
```

```
Average Income by Gender:
 Gender
Female    68504.577778
Male      71268.396226
Name: Monthly_Income, dtype: float64
Gender with highest average income: Male
```

Education category with highest average income

In [31]:
```
print("Education category with highest average income:", avg_income_edu.i
```

```
Education category with highest average income: Secondary
```

Households with monthly savings > KES 20,000

In [32]:
```
high_savings = df[df['Savings'] > 20000]
print(f"\nHouseholds with monthly savings > KES 20,000: {len(high_savings
```

```
Households with monthly savings > KES 20,000: 51
```

Summary table by Gender

In [33]:
```
gender_summary = df.groupby('Gender')[['Age', 'Monthly_Income', 'Monthly_
print("\nSummary table by Gender:\n", gender_summary)
```

```
Summary table by Gender:
                Age  Monthly_Income  Monthly_Expenditure
Gender
Female  44.822222    68504.577778         44680.955556
Male    45.698113    71268.396226         56786.811321
```

In [34]:
```
print("""
CONCLUSION
This analysis used Python (Pandas, NumPy, Matplotlib, Seaborn) to import,
manipulate, and visualize a household dataset containing ID, Age, Gender,
Monthly_Income, and Monthly_Expenditure. Missing numerical values were im
median and categorical values with the mode. Outliers in Monthly_Income w
the IQR method and removed (values outside Q1 - 1.5*IQR and Q3 + 1.5*IQR)
cleaner dataset for reliable statistics and plots. A Savings variable was
minus expenditure.

Key findings: income is right-skewed; higher education is associated with
income; gender groups differ in average income, expenditure, and age; and
households save above KES 20,000 monthly. Visualizations confirmed income
counts, and the Age-Income relationship.

Limitation: removing outliers reduces sample size and may exclude genuine
households, potentially understating inequality. Recommendation: analyze
and add more socioeconomic variables. Overall, the study shows Python's v
evidence-based household financial analysis and targeted interventions.
""")
```

CONCLUSION
This analysis used Python (Pandas, NumPy, Matplotlib, Seaborn) to import, clean, explore,
manipulate, and visualize a household dataset containing ID, Age, Gender, Education_Level,
Monthly_Income, and Monthly_Expenditure. Missing numerical values were imputed with the
median and categorical values with the mode. Outliers in Monthly_Income were detected via
the IQR method and removed (values outside Q1 - 1.5*IQR and Q3 + 1.5*IQR), producing a
cleaner dataset for reliable statistics and plots. A Savings variable was created as income
minus expenditure.

Key findings: income is right-skewed; higher education is associated with higher average
income; gender groups differ in average income, expenditure, and age; and only some
households save above KES 20,000 monthly. Visualizations confirmed income spread, education
counts, and the Age-Income relationship.

Limitation: removing outliers reduces sample size and may exclude genuinely wealthy
households, potentially understating inequality. Recommendation: analyze outliers separately
and add more socioeconomic variables. Overall, the study shows Python's value for
evidence-based household financial analysis and targeted interventions.

In [ ]: